

Curso Inicial Front-End

Clase 23: REACT: CICLO DE VIDA Y CONSUMO DE APIS

Módulo 3: Desarrollo con React.js y Ecosistema Moderno



Presentación

¿Cómo hacemos para que **React** traiga datos de una API apenas se abre la página o cómo podemos extraer y reutilizar esa lógica en toda nuestra aplicación? En esta clase conoceremos el **useEffect**, el Hook encargado de gestionar los "efectos secundarios". Entenderemos el **Ciclo de Vida** del componente (nacer, cambiar y morir) y dominaremos el array de dependencias para controlar cuándo ejecutar nuestro código. Finalmente, aprenderemos a crear nuestros propios **Custom Hooks** para encapsular la lógica de obtención de datos de manera limpia y profesional.



Objetivos

- Comprender el concepto de Efecto Secundario y el ciclo de vida de un componente.
- Dominar la sintaxis de **useEffect** y el uso crítico del array de dependencias y la función de limpieza (cleanup).

- Implementar llamadas a APIs reales sincronizadas con el montaje del componente.
- Crear y consumir **Custom Hooks** reutilizables para encapsular lógica de fetching de datos y estados.



Bloques Temáticos

- **Tema 1: Fundamentos**
 - Protocolo HTTP y Verbos (GET, POST, etc.)
 - Códigos de Estado HTTP
 - Sincronía vs. Asincronía
 - Estructura Try / Catch / Finally
 - Fetch según verbo HTTP
- **Tema 2: Hook useEffect para efectos secundarios y peticiones asíncronas (Fetch/Axios).**
 - ¿Qué es un Efecto Secundario?
 - Sintaxis y Array de Dependencias (Gatillos)
 - Función de Limpieza (Cleanup)
 - Patrón Profesional: Fetching con Async/Await
 - Tips de Debugging y Strict Mode
- **Tema 3: Custom Hooks**

- Concepto y Reglas
- Estructura Estándar
- Implementación paso a paso de useFetch
- Manejo de Estados y Lógica Asíncrona

Tema 1: Fundamentos

1. El Protocolo HTTP y sus Verbos

HTTP (*Hypertext Transfer Protocol*) es el idioma oficial de la web. Es un protocolo basado en el modelo **Cliente-Servidor**. El cliente (el navegador o tu app de React) envía un mensaje de **Petición (Request)** y el servidor responde con un mensaje de **Respuesta (Response)**.

Para decirle al servidor qué acción queremos realizar, HTTP utiliza **Verbos o Métodos**. Los más comunes son:

- **GET:** Se usa exclusivamente para **solicitar o leer** datos del servidor. No modifica nada en la base de datos y no lleva un cuerpo (**body**).
- **POST:** Se usa para **crear o enviar** nueva información al servidor (por ejemplo, registrar un usuario o crear un producto). Lleva los datos dentro del **body**.
- **PUT:** Se usa para **reemplazar o actualizar por completo** un recurso existente. Si modificas un producto, tienes que enviarle el objeto entero modificado.
- **PATCH:** Se usa para **actualizaciones parciales**. Si solo quieres cambiar el precio de un producto pero no el nombre ni la descripción, usas **PATCH**.
- **DELETE:** Se usa para **eliminar** un recurso específico del servidor.

2. Los Códigos de Estado HTTP más Comunes

Cuando el servidor te responde, incluye un código de tres dígitos que te dice qué pasó con tu petición. Se agrupan por centenas:

- **2xx (Éxito):** Todo salió bien.
- **200 OK:** La petición fue exitosa (común en **GET**).
- **201 Created:** El recurso se creó con éxito (común en **POST**).
- **3xx (Redirección):** El recurso se movió a otro lado.
- **4xx (Errores del Cliente):** Te equivocaste en algo al mandar la petición.
- **400 Bad Request:** La petición está mal armada o faltan datos.
- **401 Unauthorized:** No estás autenticado (te falta el token o la contraseña).
- **403 Forbidden:** Estás autenticado, pero no tienes permisos para ver eso.
- **404 Not Found:** Lo que buscas no existe.
- **5xx (Errores del Servidor):** El cliente hizo todo bien, pero el servidor explotó por dentro.
- **500 Internal Server Error:** Un error genérico en el código del backend.

💡 **Para memorizarlos con humor:** Si querés ver estos códigos explicados con fotos de gatos, no dejes de visitar el clásico sitio [HTTP Cats](http://HTTPCats.com).

3. Sincronía vs. Asincronía en JavaScript

JavaScript es **monohilo** (single-threaded), lo que significa que solo puede hacer una cosa a la vez.

Sincronía (Bloqueante)

Las tareas se ejecutan en orden secuencial. Cada línea debe esperar a que la anterior termine para ejecutarse. Si una tarea tarda mucho, la página se "congela".

- **Ejemplo de la vida real:** Hacer una fila en el supermercado. El cajero no atiende al cliente 2 hasta que terminó por completo con el cliente 1.
- **Ejemplo en código:**

```
console.log("Inicio");  
alert("Esto congela la pantalla"); // El usuario debe clickear OK para  
avanzar  
console.log("Fin");
```

Asincronía (No Bloqueante)

Permite arrancar una tarea que va a tardar (como pedir datos a una API) y, en lugar de quedarse esperando congelado, JavaScript la pasa a "segundo plano" y sigue ejecutando el resto del código. Cuando la tarea larga termina, avisa y se procesa el resultado.

- **Ejemplo de la vida real:** Ir a un restaurante. El mozo toma tu pedido, lo lleva a la cocina y, mientras cocinan tu plato, va a tomar el pedido de otra mesa. No se queda parado en la cocina esperando tu comida.
- **Ejemplo en código:**

```
console.log("Pido la comida (Inicio)");
setTimeout(() => {
  console.log("Comida lista (Tarea asíncrona terminada)");
}, 3000); // Tarda 3 segundos en segundo plano
console.log("Sigo charlando con mis amigos (Continuación del flujo)");
```

¿Por qué usamos **async / await**?

Históricamente, la asincronía en JavaScript se manejaba con *Callbacks* (que generaban códigos ilegibles llamados **Callback Hell**) y luego con *Promesas* (`.then().catch()`).

async / await es "azúcar sintáctico". No cambia cómo funciona JavaScript, pero nos permite **escribir código asíncrono con una estructura que parece síncrona** (línea por línea). Hace que el código sea muchísimo más fácil de leer, mantener y depurar.

4. Estructura Genérica: Bloques **try, catch y finally**

Este bloque es una estructura de control de flujo diseñada para el **manejo de errores** en cualquier lenguaje de programación, no solo para hacer **fetch**. Evita que tu aplicación se rompa y deje de funcionar cuando algo sale mal.

```
try {
  // 1. El bloque de la confianza
  // Acá pones el código "peligroso" que podría fallar por razones ajenas
  // a tu control.
} catch (error) {
  // 2. El bloque de rescate
  // Este bloque SOLO se ejecuta si algo falló dentro del 'try'.
```

```
// Recibe un objeto 'error' con el detalle de lo que se rompió.  
} finally {  
    // 3. El bloque de limpieza (Opcional)  
    // Se ejecuta SIEMPRE, pase lo que pase (haya habido error o no).  
}
```

- **try (Intentar):** Metes el código que dependa de factores externos. Por ejemplo: leer un archivo del disco duro, parsear un texto a JSON, o conectar con una base de datos. Si el archivo no existe o el JSON está mal formateado, el programa lanzará una excepción (Básicamente un error).
- **catch (Capturar):** Si el **try** falla, el programa no se muere. El flujo salta instantáneamente al **catch**. Acá defines el "plan B": mostrar un mensaje amigable al usuario, registrar el error en un archivo de logs, o setear valores por defecto.
- **finally (Finalmente):** Se usa para tareas de limpieza que deben ocurrir sí o sí. Ejemplo: cerrar la conexión a una base de datos o apagar un indicador de carga, independientemente de si la operación fue un éxito o un fracaso.

5. Cómo cambia fetch según el verbo HTTP

Por defecto, si tú haces `fetch("url")`, JavaScript asume que estás haciendo un GET. Si quieres usar cualquier otro verbo (POST, PUT, DELETE, etc.), tienes que pasarle un **segundo argumento**: un objeto de configuración.

Ejemplo: Petición GET (Por defecto)

No necesita configuración extra ni cuerpo.

```
const respuesta = await fetch("https://api.com/usuarios/1");  
const data = await respuesta.json();
```

Ejemplo: Petición POST (Crear)

Aquí necesitas especificar tres cosas clave en el objeto de configuración:

1. **method:** El verbo HTTP en mayúsculas ('**POST**').
2. **headers:** Cabeceras para decirle al servidor qué tipo de datos le estás mandando (comúnmente '**Content-Type**' : '**application/json**').
3. **body:** Los datos reales que vas a enviar, convertidos a una cadena de texto con **JSON.stringify()**.

```
const nuevoUsuario = { nombre: "Javier", rol: "Profesor" };  
const respuesta = await fetch("https://api.com/usuarios", {  
  method: "POST",  
  headers: {  
    "Content-Type": "application/json"  
  },  
  body: JSON.stringify(nuevoUsuario) // Transformamos el objeto JS a texto  
  JSON  
});  
const data = await respuesta.json();
```

Ejemplo: Petición PUT o PATCH (Actualizar)

La estructura es exactly equal a la de un **POST**, pero cambia el **method** y usualmente la URL apunta a un ID específico del recurso que quieres modificar.

```
const datosActualizados = { rol: "Full Stack Developer" };  
const respuesta = await fetch("https://api.com/usuarios/1", {
```

```
method: "PATCH", // O 'PUT' si vas a reemplazar todo el usuario
headers: {
  "Content-Type": "application/json"
},
body: JSON.stringify(datosActualizados)
});
```

Ejemplo: Petición DELETE (Eliminar)

Generalmente no requiere cabeceras ni un cuerpo (body), porque el ID del elemento que vas a borrar ya viaja directamente en la URL. Solo necesitas especificar el método.

```
const respuesta = await fetch("https://api.com/usuarios/1", {
  method: "DELETE"
});
// Muchas APIs devuelven un status 204 (No Content) en DELETE, por lo que
no hace falta hacer .json()
```

Ya sabemos que `useState` es la memoria del componente. Pero los componentes no viven aislados; a veces necesitan interactuar con el "Mundo Exterior": pedir datos a una API, conectarse a un chat en vivo, o interactuar con el DOM directo.

En **React**, a cualquier conexión con el mundo exterior se le llama **Efecto Secundario (Side Effect)**, y para controlarlos usamos el hook más poderoso y peligroso: `useEffect`.

 **Recurso Oficial:** [React Docs - Sincronizando con Efectos](#)

Tema 2: Hook useEffect para efectos secundarios y peticiones asíncronas (Fetch/Axios)

1. ¿Qué es un Efecto Secundario?

Imagina que `useEffect` es un **Sensor de Movimiento** que instalas en tu componente. Tú, como programador, programas el sensor para que ejecute una tarea específica, pero **tú también debes decirle a `React` en qué momento exacto debe encender ese sensor.**

El hook `useEffect` recibe dos parámetros:

1. **La Función (La Acción):** ¿Qué código quiero ejecutar?
2. **El Array de Dependencias (El Gatillo):** ¿Cuándo quiero que se ejecute?

Comparativa: Las 3 formas de configurar el Gatillo

Configuración del Gatillo	Código Visual	¿Cuándo se dispara la acción?	¿Cuándo usarlo?
Array Vacío <code>[]</code>	<code>useEffect(() => {...}, [])</code>	Solo 1 vez en la vida. Cuando el componente nace (se inyecta en el DOM).	Pedir datos iniciales de una API (<code>fetch</code>).
Array con Variable <code>[x]</code>	<code>useEffect(() => {...}, [x])</code>	Al nacer Y cada vez que la variable <code>x</code> cambie de valor.	Recalcular totales, guardar historial, buscar nuevos datos si el <code>id</code> cambia.
Sin Array (Ausente)	<code>useEffect(() => {...})</code>	¡Siempre! Cada vez que la pantalla titile o cambie cualquier estado por mínimo que sea.	✗ NUNCA. Congelará y arruinará el rendimiento de tu PC.

🔴 Checkpoint de Comprensión (El Bucle de la Muerte ∞): Imagina este código: `const [contador, setContador] = useState(0); useEffect(() => { setContador(contador + 1); }, [contador]);` Has puesto al estado `contador` como dependencia. Analiza la lógica paso a paso: Cuando el componente nace, el efecto se ejecuta y suma 1. Al sumar 1, el estado cambió. ¿Qué le dirá el sensor de movimiento a **React** que haga a continuación? ¿En qué momento se detendrá este proceso?

2. Limpieza (Cleanup Function)

A veces, nuestros efectos dejan procesos en segundo plano. Si abres una conexión de chat en vivo o inicias un temporizador (`setInterval`) y luego el usuario cambia de página (el componente "muere"), ¡ese temporizador seguirá corriendo invisiblemente y gastando RAM!

Para solucionarlo, el `useEffect` tiene la capacidad de retornar una función "barrendera" al final.

Ejemplo Visual de Limpieza:

```
import { useEffect, useState } from 'react';

function Reloj() {
  const [hora, setHora] = useState(new Date());

  useEffect(() => {
    // ACCIÓN: Creamos un reloj que hace "Tic" cada segundo.
    const timer = setInterval(() => {
      setHora(new Date());
      console.log("Tic");
    }, 1000);

    // LIMPIEZA: Devolvemos una función que React guardará bajo llave.
    // React ejecutará este 'return' EXCLUSIVAMENTE cuando el
    componente muera.
    return () => {
      clearInterval(timer);
      console.log("Componente destruido, reloj apagado.");
    };
  }, []); // Array vacío: Solo encender el reloj al nacer.

  return <h1>{hora.toLocaleTimeString()}</h1>;
}
```

3. Patrón Profesional: Fetching con Async/Await

Una de las grandes reglas de `useEffect` es que **no le gustan las funciones asíncronas directas** (no puedes escribir `useEffect(async () => {})`). Para usar `await` de forma limpia, debes crear la función adentro y llamarla inmediatamente.

Ejemplo de Conexión a API:

```
useEffect(() => {  
  // 1. Definimos la función asíncrona por dentro  
  const obtenerUsuarios = async () => {  
    try {  
      const respuesta = await fetch("https://api.com/usuarios");  
      const datos = await respuesta.json();  
      setEstado(datos); // Dibujamos la pantalla  
    } catch (error) {  
      console.error("Fallo de red:", error);  
    }  
  }  
};  
  
// 2. La invocamos en el acto  
obtenerUsuarios();  
}, []); // Pedimos los usuarios solo una vez al cargar la app
```

¡NOTE

El Fantasma del Modo Estricto (Strict Mode): Si estás programando y notas que tu API se llama 2 veces, o ves dos "Tic" del reloj al mismo tiempo, no entres en pánico. Es una herramienta de **React** llamada `<React.StrictMode>` que duplica los efectos intencionalmente para ayudarte a descubrir si te olvidaste de escribir tu función de "Limpieza" (`return () => {}`). Este comportamiento doble **solo ocurre en desarrollo**, en producción a los usuarios les saldrá una sola vez.

 **Tip de Debugging:** La pestaña 'Network' de la devTool en las herramientas de desarrollador es fundamental. Allí puedes ver si el error es 404, 500, o si la petición siquiera ocurre.

Checklist de resolución:

- ¿La API se llama infinitamente? → Revisa el array de dependencias.
- ¿El efecto no se dispara? → Revisa las variables de referencia.
- ¿Errores extraños? → ¿Tienes la función de limpieza?

Tema 3: Custom Hooks

1. ¿Qué es un Custom Hook y por qué lo usamos?

Un **Custom Hook** (Hook Personalizado) es una función de JavaScript que encapsula lógica reutilizable de **React** (como estado, efectos secundarios y llamadas a APIs) para poder consumirla en múltiples componentes sin repetir código.

| Reglas y Orden Natural de Nombres

Convención de Nombres: Su nombre siempre debe empezar obligatoriamente con la palabra **use** (ej: **useFetch**, **useAuth**, **useLocalStorage**). Esto permite que los linters de React detecten que es un hook y verifiquen que cumple las reglas de los hooks.

Estructura Estándar: Suelen mantener un orden natural:

- Definición de variables de estado **iniciales**.
- Implementación de funciones de lógica de negocio o **asincronismo** (**Try/Catch/Finally**).
- Ejecución de efectos secundarios internos (**useEffect**).
- Retorno de las variables y funciones esenciales de control (**data**, **loading**, **error**, **refetch**).

Lógica de Fetching, Estados de Control y Verbos HTTP

En un Custom Hook para APIs (**useFetch**), definimos tres estados fundamentales:

- **data** (el contenido retornado de la API, por defecto null).
- **loading** (indica si la petición está en curso, por defecto true).
- **error** (almacena el mensaje o el objeto del fallo, por defecto null).

Bloque Asíncrono de Petición

Para realizar la llamada asíncrona, encapsulamos la petición en un bloque try...catch...finally. El bloque finally se ejecuta al terminar la llamada sin importar si fue exitosa o fallida, permitiendo apagar el cargador (**setLoading(false)**).

Implementación Completa

```
import { useState, useEffect } from "react";

function useFetch(url, opciones = {}) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  const realizarPetición = async () => {
    setLoading(true);
    setError(null);
    try {
      const respuesta = await fetch(url, opciones);

      if (!respuesta.ok) {
        throw new Error(`Error en la llamada: ${respuesta.status} ${respuesta.statusText}`);
      }
    }
  }
}
```

```
const datosJson = await respuesta.json();
setData(datosJson);
} catch (err) {
  setError(err.message || "Ocurrió un error en la API");
} finally {
  setLoading(false);
}
};

useEffect(() => {
  realizarPetición();
}, [url]);

return { data, loading, error, refetch: realizarPetición };
}

export default useFetch;
```

Explicación Detallada

¿Qué sucede en este Custom Hook? Este hook automatiza el ciclo de vida completo de una petición HTTP (la espera, el éxito y el fallo) para que no tengas que repetir esta lógica en cada componente.

1. Inicialización del Estado

Al principio del hook se definen tres estados fundamentales

data (null): Almacenará la respuesta exitosa de la API cuando llegue.

loading (true): Empieza en true porque asumimos que en cuanto el componente se monte, la petición va a arrancar inmediatamente.

error (null): Almacenará el mensaje de error si algo sale mal.

2. La función realizarPetición

Es una función asíncrona (**async/await**) que se encarga de hacer el trabajo sucio. Lo primero que hace al ejecutarse es "resetear" el estado: pone **loading** en **true** y **error** en **null**. Esto es vital por si la función se vuelve a ejecutar más de una vez (**refetch**).

El bloque try:

Ejecuta **fetch(url, opciones)**. Al pasarle el objeto opciones por parámetro, este hook no se limita solo a traer datos (**GET**), sino que es flexible: permite configurar métodos como **POST**, **PUT** o **DELETE**, enviar cabeceras (**headers**) o un cuerpo de datos (**body**).

Validación crucial (respuesta.ok): fetch no lanza un error de forma nativa si el servidor responde con un estado de error (como un 404 o un 500). Solo falla si hay un problema de red. Por eso, evaluamos **!respuesta.ok**. Si el estatus no está en el rango de los 200, forzamos un error saltando directamente al catch. Considerando que “**respuesta.ok**” es del tipo booleano, puede retornar true o false.

Si todo está bien, parsea el **JSON** (Convierte el dato desde JSON a Objeto de javascript) y guarda el resultado en data mediante **setData**.

El bloque catch:

Si la red se cae o el if (**!respuesta.ok**) se dispara, el flujo cae acá. Captura el mensaje de error y lo guarda en el estado para que el componente sepa qué falló y pueda mostrar una alerta en la UI.

El bloque finally:

Pase lo que pase (**éxito o error**), este bloque se ejecuta siempre al final. Apaga el estado de carga (**setLoading(false)**), asegurando que la interfaz deje de mostrar el spinner o el texto de "Cargando...".

3. El Disparador (**useEffect**):

El **useEffect** se encarga de ejecutar **realizarPetición()** de manera automática. Tiene como dependencia **[url]**.

Esto significa que:

Se ejecutará la primera vez que el componente se monte en la pantalla.

Si durante la vida del componente la url cambia (por ejemplo, pasamos de /productos/1 a /productos/2), el efecto se vuelve a disparar automáticamente, trayendo los nuevos datos.

4. El Retorno

El hook devuelve un objeto con cuatro elementos:

- **data, loading y error** para que el componente renderice la interfaz según el estado actual.
- **refetch: realizarPetición**: Exporta la función renombrada como **refetch**. Esto es una excelente práctica; le permite al componente volver a disparar la petición de forma manual (por ejemplo, al hacer clic en un botón de "**Reintentar**" o "**Actualizar datos**") sin necesidad de que cambie la URL.

Documentación Oficial (Referencias)

- React: <https://es.react.dev/>